

Flujos del sistema

Esta sección documenta los flujos más importantes de Savorealo: autenticación, ciclo de vida de una request, generación de recetas con IA, feeds, mensajería en tiempo real, subida de imágenes y CI/CD.

1. Flujo de autenticación

```
sequenceDiagram
    actor U as Usuario
    participant C as Cliente (Web / Mobile)
    participant S as Supabase Auth
    participant W as Worker GraphQL
    participant DB as PostgreSQL

    U->>C: Introduce email + contraseña (o Google Sign-In)
    C->>S: supabase.auth.signInWithPassword(...)
    S->>S: Valida credenciales
    S-->>C: { access_token (JWT), refresh_token }
    Note over C: Web: guarda en localStorage<br/>Mobile: persistencia Supabase

    C->>W: POST /graphql\nAuthorization: Bearer JWT
    W->>W: verifyJWT(token, SUPABASE_JWT_SECRET)\nHMAC-SHA256 - sin llamada ex
    W->>W: Extrae { userId, email } del payload
    W->>DB: Prisma query/mutation con userId en contexto
    DB-->>W: Datos
    W-->>C: Respuesta GraphQL
```

Renovación automática del token

```
flowchart LR
    Token["JWT expira\n(por defecto 1h)"]
    SDK["Supabase SDK\n(JS o KT)"]
    Refresh["POST /auth/v1/token\ngrant_type=refresh_token"]
    New["Nuevo JWT"]
    Retry["Reintenta request original"]

    Token --> SDK
    SDK --> Refresh
    Refresh --> New
    New --> Retry
```

```
Token → SDK
SDK → Refresh
Refresh → New
New → Retry
```

2. Flujo de request completo

```
flowchart TD
    C["Cliente\nAngular / Android"]

    subgraph Edge ["Cloudflare Edge (Worker)"]
        CORS["Validación CORS\nOrigin permitido?"]
        Route["Routing"]
        Health["/health → JSON status"]
        Auth["Auth Middleware\nverifyJWT()"]
        Ctx["Contexto GraphQL\n{ prisma, userId }"]
        Resolver["Pothos Resolver\nquery / mutation"]
        ErrH["Error Handler\nerrors.ts → mensaje amigable"]
    end

    subgraph Data ["Datos"]
        Acc["Prisma Accelerate\n(connection pooling)"]
        PG["PostgreSQL\nSupabase"]
    end

    C -->|POST /graphql + Bearer JWT| CORS
    CORS --> Route
    Route -->|/health| Health
    Route -->|/graphql| Auth
    Auth --> Ctx
    Ctx --> Resolver
    Resolver --> Acc --> PG
    PG --> Acc --> Resolver
    Resolver --> C
    Resolver -->|error| ErrH --> C
```

3. Flujo de generación de recetas con IA

```
sequenceDiagram
    actor U as Usuario
    participant C as Cliente
    participant W as Worker GraphQL
    participant DB as PostgreSQL
    participant AI as Agente IA (Webhook externo)

    U->>C: Escribe prompt: "Paella para 4 personas"
    C->>W: mutation generateRecipe(message: "...")
    W->>DB: INSERT ai_generations { userId, prompt, status: PENDING }
    W->>AI: Webhook asíncrono (fire-and-forget)
    W-->>C: { started: true }
    Note over C,W: Respuesta inmediata - el cliente no espera

    loop Polling (query myGenerations)
        C->>W: query myGenerations
        W->>DB: SELECT WHERE userId AND status IN (PENDING, DONE)
        W-->>C: [{ id, status: PENDING, ... }]
    end

    AI->>AI: Procesa con LLM
    AI->>DB: UPDATE ai_generations SET status = DONE, result = { pasos, ingred:

    C->>W: query myGenerations
    W-->>C: [{ id, status: DONE, post: null }]
    U->>C: Acepta receta generada
    C->>W: mutation createRecipePost(generationId)
    W->>DB: INSERT posts + recipes + UPDATE ai_generations.converted_to_post_id
    W-->>C: Post publicado
    Note over C: Post aparece en el feed
```

Estados de generación

Estado	Icono	Descripción
PENDING	Procesando	El agente IA está generando la receta
DONE	Listo	Receta generada, disponible para publicar
FAILED	Error	Falló la generación — se puede reintentar

4. Flujo del feed principal (Following)

```

flowchart TD
    Open["Usuario abre el Feed"]
    subgraph Query ["query feed(first: 10)"]
        Filter["WHERE author_id IN following_ids\nAND created_at > NOW() - 7 d"]
        Order["ORDER BY created_at DESC"]
        Page["LIMIT 10 AFTER cursor"]
    end
    Response["edges[] + pageInfo\n{ hasNextPage, endCursor }"]
    Scroll["Usuario hace scroll hasta el final"]
    Next["query feed(first: 10, after: endCursor)"]

    Open --> Filter
    Filter --> Order
    Order --> Page
    Page --> Response
    Response --> Scroll
    Scroll --> Next
    Next --> Filter

```

- Paginación **Relay** con cursores (no offset) — evita posts duplicados/saltados
- `liked` y `saved` devueltos por post: el cliente sabe el estado sin queries extra
- `imageUrl`: primera imagen del post (`position: 0` en `post_media`)

5. Flujo del feed Descubrir (algoritmo de scoring)

```
flowchart LR
    subgraph RPC ["PostgreSQL RPC: get_personalized_feed(userId, limit, offset)"]
        E["Engagement score\nlikes + comments + saves"]
        A["Affinity score\npreferencias culinarias del usuario"]
        F["Freshness score\npenalización por antigüedad"]
        Score["score = E + A + F"]
    end

    Query["query discoverFeed(limit: 20, offset: 0)"]
    Result["Posts ordenados por score DESC"]
```

```
Query → RPC
E → Score
A → Score
F → Score
Score → Result
```

6. Flujo de mensajería en tiempo real

```
sequenceDiagram
    participant A as Usuario A (remitente)
    participant SB as Supabase Realtime
    participant DB as PostgreSQL
    participant B as Usuario B (receptor)

    Note over A,B: Ambos suscritos al canal de la conversación via WebSocket
    A->>SB: INSERT direct_messages { conversation_id, sender_id, content }
    SB->>DB: Persiste el mensaje
    SB-->>B: Evento Realtime: nuevo mensaje
    Note over B: Recibe el mensaje instantáneamente sin polling
    B->>SB: INSERT direct_messages { content: "respuesta" }
    SB-->>A: Evento Realtime: respuesta recibida
```

La mensajería **no pasa por el Worker GraphQL** — usa Supabase Realtime directamente para latencia mínima.

7. Flujo de subida de imagen y creación de post

```
sequenceDiagram
    actor U as Usuario
    participant C as Cliente (Web/Mobile)
    participant SS as Supabase Storage
    participant W as Worker GraphQL
    participant DB as PostgreSQL

    U->>C: Selecciona/captura imagen
    Note over C: Web: FileUpload → ArrayBuffer<br/>Mobile: CameraX → Bitmap
    C->>SS: upload(bucket, path, imageData)
    SS-->>C: URL pública: https://...supabase.co/storage/v1/object/public/...
    C->>W: mutation createPost(title, content, imageUrl)
    W->>DB: INSERT posts { title, content, post_type: PHOTO }
    W->>DB: INSERT post_media { media_url, media_type: IMAGE, position: 0 }
    W-->>C: Post creado con ID
    Note over C: Post aparece en el feed de seguidores
```


8. Flujo de traducción de contenido

```
sequenceDiagram
    actor U as Usuario
    participant C as Cliente
    participant CTS as ContentTranslationService
    participant Cache as Caché en memoria (Map)
    participant API as MyMemory API (pública)

    U->>C: Clic en [Traducir]
    C->>CTS: translate("Ciao mondo")
    CTS->>Cache: ¿Existe clave "en::Ciao mondo"?
        alt Caché HIT
            Cache-->>CTS: "Hello world"
        else Caché MISS
            CTS->>API: GET /get?q=Ciao mondo&langpair=auto|en
            Note over API: Sin API key · Detección automática del idioma origen
            API-->>CTS: { translatedText: "Hello world" }
            CTS->>Cache: Guarda "en::Ciao mondo" → "Hello world"
        end
        CTS-->>C: "Hello world"
    C->>C: Renderiza texto traducido
    Note over C: Si idioma activo = 'ar' → aplica dir="rtl" al contenedor
```

9. Flujo de CI/CD

```
flowchart TD
    Push["git push → GitHub"]

    subgraph CI ["GitHub Actions"]
        subgraph FE ["Frontend (Angular)"]
            FI["npm ci"]
            FL["npm run lint"]
            FT["npm test (Vitest)"]
            FB["npm run build:cloudflare"]
            FD["Deploy → Cloudflare Pages"]
            FI → FL → FT → FB → FD
        end
    end

    subgraph BE ["Backend (Worker)"]
        BI["npm ci"]
        BT["npm test (Vitest)"]
        BD["npx wrangler deploy"]
        BI → BT → BD
    end

    subgraph MOB ["Mobile Android"]
        MT["./gradlew test"]
        MB["./gradlew assembleRelease"]
        MA["Artefacto APK/AAB"]
        MT → MB → MA
    end

    Push → FI
    Push → BI
    Push → MT
```

Resumen de flujos

Flujo	Protocolos	Latencia típica
Autenticación	HTTPS + Supabase Auth	~300ms
Request GraphQL	HTTPS + JWT	~100ms (edge)
Generación IA	GraphQL (fire-and-forget) + Webhook	5–30s async
Feed following	GraphQL + Prisma + PostgreSQL	~150ms
Feed descubrir	GraphQL + PostgreSQL RPC	~200ms
Mensajería	WebSocket (Supabase Realtime)	<100ms
Subida imagen	HTTPS multipart → Supabase Storage	~500ms–2s
Traducción	HTTPS → MyMemory API + caché local	<50ms (caché) / ~400ms (API)